

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE ENSINO SUPERIOR DO SERIDÓ
DEPARTAMENTO DE CIÊNCIAS EXATAS E APLICADAS
PROGRAMA DE GRADUAÇÃO EM SISTEMAS DE INFORMAÇÃO

Ariadny Francisca Dantas Santos

José Salustiano Neto Junior

Riam Stefesom Venancio da Silva

Oficina Mecânica: Projeto Arquitetural do Software

Caicó – RN

2026

SUMÁRIO

Projeto Arquitetural do Software - Sistema de Gestão de Oficina Mecânica

1. Introdução

2. Visão Geral da Arquitetura

2.1. Camada de Frontend (Apresentação)

2.2. Camada de Backend (Regras de Negócio e API)

2.3. Camada de Persistência (Dados)

3. Mecanismos Arquiteturais

4. Requisitos Não-Funcionais (RNF)

4.1. RNF01 - Segurança e Controle de Acesso

4.2. RNF02 - Integridade Referencial e Exclusão Lógica

4.3. RNF03 - Desempenho e Validação Automatizada

5. Estrutura de Componentes e Módulos

5.1. Módulo de Autenticação e Usuários

5.2. Módulo de Clientes

5.3. Módulo de Veículos

5.4. Módulo de Insumos

5.5. Módulo de Procedimentos

5.6. Módulo de Ordens de Serviço (OS)

6. Arquitetura de Implantação (Deployment)

Projeto Arquitetural do Software - Sistema de Gestão de Oficina Mecânica

1. Introdução

Este documento descreve o Projeto Arquitetural do Software para o Sistema de Gestão de Oficina Mecânica. O objetivo é apresentar as diretrizes de design, os padrões arquiteturais adotados, os mecanismos técnicos de implementação e a estruturação dos módulos do sistema, servindo como guia de referência técnico após a conclusão do desenvolvimento do software.

O sistema foi projetado para automatizar os fluxos operacionais de uma oficina mecânica de pequeno porte, garantindo o controle rigoroso sobre os cadastros de clientes, veículos, insumos, procedimentos e a emissão de ordens de serviço, além de implementar uma rígida política de controle de acesso baseada em perfis de usuários.

2. Visão Geral da Arquitetura

O sistema adota o padrão arquitetural de **Aplicações Desacopladas** (Decoupled Architecture), utilizando o modelo cliente-servidor baseado em uma API RESTful. A separação clara entre as responsabilidades de interface e as regras de negócio confere ao sistema alta manutenibilidade, escalabilidade e performance.

2.1. Camada de Frontend (Apresentação)

Desenvolvida em **React.js** utilizando componentes funcionais, a interface opera como uma Single Page Application (SPA). É responsável por gerenciar as rotas internas de navegação no navegador, renderizar as telas dinamicamente de acordo com o estado da aplicação e impor as restrições visuais baseadas no nível de permissão do usuário logado (armazenado com segurança no estado local).

2.2. Camada de Backend (Regras de Negócio e API)

Desenvolvida em **Django** utilizando o **Django REST Framework (DRF)**. O backend segue o padrão de arquitetura Model-ViewSet-Serializer. É responsável pelo processamento lógico de dados, validação de integridade através dos serializers, segurança e controle de acesso via tokens de autenticação (JWT) e exposição dos endpoints REST que expõem os dados em formato JSON.

2.3. Camada de Persistência (Dados)

Utiliza o Sistema Gerenciador de Banco de Dados (SGBD) relacional **PostgreSQL**. A persistência de dados e o mapeamento relacional de objetos são gerenciados de forma automatizada pelo ORM (Object-Relational Mapping) nativo do Django, garantindo transações seguras e o isolamento das tabelas.

3. Mecanismos Arquiteturais

A tabela abaixo mapeia as decisões de design da arquitetura, relacionando as necessidades de análise com os conceitos de design e as ferramentas reais de implementação utilizadas no encerramento do projeto:

Mecanismo de Análise	Mecanismo de Design	Mecanismo de Implementação
Persistência de Dados	Banco de Dados Relacional (SGBD)	PostgreSQL e Django ORM
Autenticação de Usuários	Autenticação via Token baseada em Estado	Tokens de Acesso JWT (JSON Web Tokens)
Controle de Autorização	Controle de Acesso Baseado em Perfis (RBAC)	Custom Permissions no DRF e travas condicionais no React
Comunicação de Dados	Serviços Web Baseados em Rotas REST	Biblioteca Axios no Frontend e JSON na API
Preservação de Histórico	Exclusão Lógica e Suspensão de Cadastros	Flag booleana is_active nas tabelas do banco

4. Requisitos Não-Funcionais (RNF)

4.1. RNF01 - Segurança e Controle de Acesso

O acesso ao sistema é estritamente restrito através de uma tela de login obrigatória. O sistema implementa dois níveis hierárquicos de usuários bem definidos nas regras de negócio:

- **Administradores:** Possuem privilégios totais sobre a plataforma. São os únicos autorizados a criar novos funcionários, gerenciar permissões administrativas (promover ou remover o status de admin) e acessar a tela "Gerenciar Equipe". Possuem acesso exclusivo às rotas de edição e exclusão de todas as entidades do sistema.
- **Funcionários:** Possuem acesso operacional limitado. Estão autorizados unicamente a visualizar listagens, consultar detalhes e cadastrar novos Clientes, Veículos, Procedimentos, Insumos e Ordens de Serviço. O sistema bloqueia nativamente o acesso deles à tela de gerenciamento de equipe e às funções de alteração e exclusão de registros.

4.2. RNF02 - Integridade Referencial e Exclusão Lógica

Para impedir a quebra de histórico de ordens de serviço antigas ou vínculos operacionais, o sistema não realiza exclusões físicas (comandos SQL DELETE) para entidades centrais como Clientes. Em vez disso, utiliza a exclusão lógica controlada por atributos booleanos ativos/inativos, ocultando registros deletados das buscas convencionais, mas mantendo o relacionamento histórico intacto no banco de dados.

4.3. RNF03 - Desempenho e Validação Automatizada

Toda a higienização de dados e checagem de tipos ocorre na camada de serialização do backend antes de atingir o banco de dados. Isso reduz requisições inválidas e otimiza o tráfego de dados, garantindo respostas rápidas nas operações diárias da oficina mecânica.

5. Estrutura de Componentes e Módulos

O sistema está estruturado em seis módulos independentes que conversam de forma coordenada através do mapeamento relacional:

5.1. Módulo de Autenticação e Usuários

Controla as sessões, valida as credenciais de login e injeta o cabeçalho de autorização nas requisições HTTP do frontend. Centraliza a regra de negócio que restringe a manipulação da tabela de colaboradores aos administradores cadastrados.

5.2. Módulo de Clientes

Gerencia os dados cadastrais das pessoas físicas. O campo CPF foi estabelecido como a Chave Primária (Primary Key) do modelo, impedindo duplicidade no banco. Em casos de reativação de um cliente inativo, o sistema herda os dados antigos do banco e os atualiza automaticamente via requisições do tipo PATCH.

5.3. Módulo de Veículos

Armazena as propriedades dos veículos (Placa, Marca, Modelo, Tipo, Cor e Ano). Apresenta um relacionamento do tipo chave estrangeira (Foreign Key) obrigatório com a tabela de Clientes, garantindo que nenhum veículo fique órfão no sistema.

5.4. Módulo de Insumos

Controla as peças, óleos e materiais utilizados na oficina. Fornece o catálogo de itens físicos que podem ser associados às manutenções em andamento.

5.5. Módulo de Procedimentos

Padroniza a precificação da mão de obra da oficina, registrando o nome do serviço, o valor cobrado e a estimativa de tempo médio de execução.

5.6. Módulo de Ordens de Serviço (OS)

O módulo agregador do sistema. Concentra os vínculos de veículos e procedimentos para abrir ordens de manutenção, calculando valores totais e controlando o fluxo do ciclo de vida de cada atendimento gerado na oficina mecânica.

6. Arquitetura de Implantação (Deployment)

A distribuição do software concluído foi planejada de forma modular no ambiente físico/nuvem:

1. O artefato gerado na build do frontend React.js é distribuído estaticamente em servidores CDN otimizados para entrega rápida de interfaces web.
2. O container contendo a API Django REST Framework é executado de forma isolada em servidores de aplicação baseados em nuvem, recebendo e processando apenas requisições HTTP REST vindas do frontend.
3. O SGBD PostgreSQL roda de forma centralizada e isolada do acesso público direto, aceitando requisições de conexão estritamente autenticadas vindas da instância ativa do backend Django.